lecture-week3-rec_compressed.mp4 transcript with timestamps

00:00
Alright, this is our lecture week 3. In this lecture we're going to talk about data visualization by painting and singing. If you believe me, we'll find out. So we're going to talk about two libraries or modules in Python that are used for data visualization, commonly used. One is Pandas. So we specifically focus on Pandas plot function. And there's another module, Seaborn, that is also used for visualization. There are also other libraries or modules which we're not going to discuss in this lecture, but we will mention them. But before we talk about the specific functions and different ways of data visualization, why do we need data visualization? We need that to understand the data, understand any meaningful patterns in the data, understand, identify any outliers, essentially identify any interesting patterns in our data. And that's an easy way to do that. It's much easier than going through the data and checking every data entry, looking at the values, going back and forth between the records of the data. With the visualization, we can have a quick look and see sometimes some really interesting patterns in our data. It can also help us, of course, to find any relationships, which is a kind of pattern that we can find in the data. So relationships between, for example, two variables.

01:43
Data visualization libraries used in Python include Seaborn, Pandas, Matplotlib, which is kind of the foundation for Seaborn and Pandas. In other words, Pandas and Seaborn use functionalities provided by Matplotlib to provide their own specific functionalities. There are also more advanced libraries or modules such as Bokeh or Plotly or Galter and others. These are more interactive visualizations, allow for user interaction with the graphs and they produce higher quality visualizations. And they're also more sophisticated to use usually. Sometimes it's hard to find how to do things with these modules, because they have a smaller communities that use them, but they're significantly better than the conventional C1 and Pandas in terms of visualization. They give you more options and produce better quality graphs or plots.

02:52
So let's start with data visualization with Pandas. As usual, we start with getting the data. So we read the data into the memory and then we start working with it. Here we

use a PD, PD is a reference or a handle to the pandas data frame, and then we call in the read pql function that reads mortality underline prepped, the pkl, which is a pql file stored, for example, on your local drive. And now we are reading it into the memory, we are deserializing it. So this is a data frame that has been serialized, stored in the data frame in the hard drive and now we are reading this what you call a data frame which has been stored on the hard drive back to the memory and we are recreating the data frame from it. That's a deserialization. So this is a first data frame and there's another data frame which is the wide version of the original data frame. It's a wide data frame with the index of year and the columns that correspond to different age groups. That's our second data frame. So the second data frame is created from mortality underline wide. It's a wide representation of the original data frame, includes all the death rates corresponding to as I said different years and different age groups where the years are the indices of your data frame. We're gonna use these two data frames for the purpose of different visualizations.

04:31
As usual we start with the simpler things in this case simpler visualization or which is a line plot as if the line, is created from mortality-wide data frame, where the columns represent different age groups and the rows are different years. Mortality-on-line-wide.plot is going to create a line plot. But this line plot is going to contain multiple lines where each line corresponds to a specific age group and as you can see they are specified in the legend this is called legend so it's got a title age group and it denotes different age groups by different colors and you can see that the blue color here specifies age group of 1 to 4 and so on and so forth. So the line plot here shows us different trends. It shows us the trends for different age groups, the trends of death rates for different age groups, and you can see that in general the death rate is decreasing over years for different age groups. So we can see the relationship between death rate and year and we can see the trend. That is a decreasing trend. And we can also see a pattern here. We can see there's a spike here around closer to 1920. There's a spike. You see the spike? It's happened across all different age groups which could be indicative of some kind of an issue or some event which might have triggered this spike as we will see later.

06:28
Here's another example of a line plot. This time we're creating a line plot for mortalit

y data. It's not mortality wide, it's mortality data. So that's the long, long data frame. And the data frame is created, sorry, the line plot is created for different columns in t he data frame. That includes year, death rate, and mean center. Each column is denot ed here by, or each attribute is denoted here by a different color. And the function w e're using here is the plot function. And the kind of the plot is set to be line so this is going to create a line plot. mortality underlined data.plot is going to create a line plo t for all the columns and the columns are year, death rate. So here's a line plot with i mproved appearance. We can do things that make the graphs more informative, esp ecially if we want the audience to get a quick understanding of the graph or the plot by just simply looking at it without having to refer to the description of the plot or th e graph. And here we can see some additional information that are added to the plo t such as the title, we have the title of the plot, we have labels for X and Y axis, so thi s is the X axis, the horizontal axis, the vertical axis represents, it's referred to as Y axis, and the label for the Y axis is deaths per 100,000. And the grid is set to true, so it's cr eating a grid that makes it easier to find the numbers or the values on the plot and s etting the rotation parameter to 45 degrees, which is going to rotate the labels of th e X sticks to 45 degrees.

08:43

Here we have another example of improving the appearance of a line plot. We're cre ating a line plot from mortality-wide data frame. We're setting the title, Y label, figur e size, grade rotation. The X limit specifies the range on the x-axis that needs to be d isplayed. Here we are only focusing on this particular range between 200 to 2018. Th at means every other data point that is before 200 or after 2018 will be excluded fro m the visualization. This is particularly useful if you want to focus on a specific range and to reduce the confusion or you want to kind of zoom in to find some specific pa tterns around certain years, for example, or any range of variables that you have. any range of variables that you have. Again, you can see that there are multiple lines on t his line plot. Each of them correspond to a different age group. And there are, what do you call it? Distinguished by different colors in the legend. Here's an exercise for you. Ask chat GPT to see how you can rotate the Y axis labels. These exercises are to encourage you to use ChatGPT and I cannot emphasize more on the importance of u sing ChatGPT for learning and improving programming skills.

10:29

Here's another example of visualization. This plot is referred to as scatter plot. It's particularly useful when there is less continuity in our data points. We don't have enough data points that create a line. We could use the scatter plots, but otherwise they're very similar to line plots. They show the relationship between two variables and also they show the trend. So in many cases, we can choose either to visualize our data using a line plot or a scatter plot, it's a choice. So the next plot we wanna talk about is area plot. It looks like a line plot but it's not a line plot. It still shows the trends like in the line plot but it also shows the volume of the data or the total magnitude of the data. Therefore it is important, it is useful when the volume or the magnitude of the data is as important as the trend. In such cases we use an area plot. We can see that by comparing these two graphs provided in this slide, you can look at the line plot that we created earlier from the same data frame, mortality-wide, and you can see the distribution of all the death rates for different age groups across different years. Take for example year 190. For this particular year, for the age group of 15 to 19 years, the death rate is around 500. And if you look at the area plot you can see that again for this particular year 190 and this age group of 15 to 19 there is a range here. This range shows the size of the death rates that correspond to this particular age group 15 to 19 in that particular year 190. So it shows the proportion of the contribution of this particular age group 15 to 19 to the death rates in that particular year in comparison with other age groups and see that you see that in comparison and the total value here is the summation of the value of all the death rates corresponding to different age groups so that is 300 something. That's the summation of all death rates but the range here shows the size of the death rate that correspond to this particular age group 15 to 19 which is again around 500 as in the original graph you see it starts around here let's say somewhere around 2750 and it goes all the way up to let's say somewhere around 2,750 and it goes all the way up to, let's say 3,250, something like that. So it's again 500, but it also shows the total number of, total number of this is, oh, to the amount of this is in that particular year, which is 3, is 300 say 250.

13:48

Bar plot is the next plot we want to discuss is particularly useful for showing changes over time so it again shows the trends and that also is useful for comparing data across different groups. So it's useful for categorical data. We have, for example, in this case, we are creating a bar plot for data points in our mortality-wide data frame. So the columns in the mortality-wide data frame are different age groups and the rows a

re years. We're only interested in these two particular years, 190 and 200. So this is a tuple, it's not a list. So we select a subset of the data frame that concerns only these two years and then we're creating a bar plot of this data frame. So it creates for us a bar plot with two, for two years where each, where within each year, we have bars th at correspond to each age group. This is what we get when we call the bar plot for th is data frame. for this data frame. Sometimes you may want to rotate your plots and make them horizontal.

15:14
Histograms are another type of graphs or plots that help us get an overview of distri bution of random variables or whatever variables that we have and they're easy to u nderstand they show the frequency of the occurrences of different ranges of values f or the parameter of interest or variable of interest in this case death rate. So you can see that there are different ranges and these ranges are specified by this parameter beans here. Beans equals 8 specifies the number of the beans here, number of categ ories. So the whole range is from 0 to 200, this is the years. This range is divided into eight sections, each section is referred to as a bin. So the width of each bin therefore would be around 250 and you can see the frequency of the occurrences, the values t hat fall within each range. So for example for the values that or for the death rates th at fall between 0 to 250 the frequency is somewhere here somewhere around 300, 6 0, 70. So this range is the most frequent or contains the most frequent death rates in our data points. So we have a data frame, in our data frame there are multiple data p oints, each data point has a death rate value associated with it and the death rates th at are frequently appeared in our data frame fall here, fall between 0 to 250 and ther e are still a notable number of this rates that fall between 250 and 500 but not many fall into other bins. So we can safely assume that nearly all this rates fall between 0 t o 500 in this case, the majority of the straights and all of those at least fall within the range of 0 to 500 shows the distribution of data. It also shows our data is, our death rates, it shows the distribution of death rate and also shows that the threat is, that all the distribution of the threat is skewed to the right. So this graph is skewed to the ri ght. So this graph is a skewed to the right and which means most points are what yo u call it on the left side of the plot.

01:48
Density plot is another plot that we can use or we can utilize to visualize the distribut

ion of our variables in this case death rate again using the plot that this density funct ion just creates a density function of the death rate. The density function is the prob ability density function or pdf. It gives us the chances of certain values occurring in o ur data or in our data frame. For example, the value that corresponds to 500, someth ing here, shows the chances or the probability that there is a data point in our data fr ame or in our dataset with 500 deaths. This shows the probability of different values occurring. As you can see, this probability is quite low. As you get closer to zero, the probability increases. And as the number of disk rates increase, the chances of them occurring in the data frame or among your data points decreases. In general, there a re fluctuations, but in general there's a trend. So again this shows you the distributio n of data, but in a different way. The distribution of the values for your particular, for your variable of interest in this case is different in a different way. So here's another e xample of a density function, density plots but instead of one value we are showing multiple values. We are again showing the death rates but for different age groups. S o you have the distribution of death rate across different age groups and each plot h ere shows or represents a different age group.

20:35
The one colored blue showing the... A very useful plot that is often used especially in research is referred to as box plots. It's very informative you can just look at it and ge t lots of information. You can see where is the first and second and third quartile of t he data. The first quartile is the median of the first half of your data when you sort y our data in, let's say, ascending order. So the first half of data, the median of the first half would be your first quartile. And then the second quartile is the median of your data. And the second quartile is the median of your data and the third quartile is the median of the second half. So the Q1 is the median of the first half, Q3 is the median of the second half and Q2 is the half, is right in the middle. And you can see the distr ibution of your data. You can see a lot of the points are coming after the median and a lot of the points have values that come after the median and in this case for examp le and of course it could be different for different for different age groups. Another i nteresting thing about boxplot is that you can visualize distribution of the data for di fferent groups in this case different age groups it also shows you the outliers based on interquartile range so interquartile range is computed based on q3 and q1 yeah q 3 minus q1 gives you the interquartile range and 1.5 times interquartile range or IQR will specify a point beyond which any point would be an outlier. So in this case this is

your 1.5 times IQR plus Q3. So Q3 plus 1.5 IQR will specify the maximum acceptable value in a way. Anything beyond that, you're considering that as an outlier. So all the se are the circles here, black circles here are outliers. Similarly there's an outlier here, there's an outlier here, there's almost an outlier here. And the whiskers extending fro m a box represent the range of values within 1.5 times the IQR. So if there's any valu e that, so the maximum value that falls within this range of Q3 plus 1.5 IQR, that wou ld be the maximum of your data and anything beyond that would be the outlier. And anything before or smaller than Q1 minus 1.5 IQR will also be an outlier, which is irre levant in this example. We don't have any values smaller than what I just mentioned. So we only have outliers that are beyond Q3 plus 1.5 times IQR.

23:43
So believe it or not, there is a song about interquartile range, which I found on YouT ube, and it's a really good summary of what is RPO-R. And we can listen to it, and yo u can sing it if you like. And here's a challenge for you. Can you compose a song for data science or AI? That would be a challenge for you if you can do that and it's a go od song. I promise I'll post it on CloudDeakin with your permission and yeah show y our talent. So here's a poem made by William Shakespeare about data science. I'm jo king. This is made by AI, ChatGPT to be more specific. Let's see if you can sing it or at least read it. We start with a question, something to solve. Dig through the data, let t he mysteries resolve. Clean it, pre-process until it's refined, seeking the patterns that are hidden inside. Data, oh data, tell us your truth. Algorithms humming, they're on t he move. Models and training accuracy is key. Prediction insights set the knowledge free. Visualize the findings in a graph or a chart. Communicate results, that's the real art. Iterate, improve, let's run it again. Data science and I on this journey we embark. Machine learning, stats and AI combine, turning raw numbers into something divine. From Python to R, tools in our hand, we're data detectives making it stand. So R is an other programming language that is used also for data science and AI. Data oh data, tell us your truth. Algorithms humming, they're on the move. Models and training, ac curacy is key. Predictions and insights, set the knowledge free. So here's to the data, our treasure trove. Which reminds me of social media. In a world of numbers, there's always more to know. Keep querying, coding, exploring the unseen. Data science an d I are a formidable team.

02:37

Okay, I hope that brought some good memories from kindergarten and now let's ge t serious again. good memories from kindergarten and now let's get serious again. L et's talk about pie chart. Pie chart displays the relative sizes of different categories or parts of the whole as slices of pie. Commonly used to show the composition of categ orical variable and are useful for conveying simple and easily understandable inform ation. Used in business, finance, and marketing to show sales figures, market share, a nd budget allocations. Partials can be less effective usually when the number of the c ategories start to grow, get larger, and it becomes less useful. Like any other graph o r plot it has its own advantages and disadvantages. A simple way to create a pie char t and it is to call the pie function from the plot so again we're using the plot function of the pandas plot.pie is going to create a pie chart for us. Here we have grouped ou r data based on the age group. These are the categories here and we are calculating the sum of death rates within each category which means within each age group and we're displaying that in the pie chart.

27:44
And here's how we can create subplots. That means when one plot gets too busy an d you want to break it into multiple subplots, to make it more visible. That's the main purpose usually. How to do that? See here we have a line plot again. So plot the line is going to create a line plot again. So plot the line is going to create line plot, but w e're going to break this line plot into four subplots. Each subplot has its own title, wh ich are listed here in the title, title is list. There's a list that is going to be used to extr act the titles of the subplots. And it's passed as a parameter of the line function or pl ot function of the panels. And you have of course other parameters set which I'm goi ng to skip. Legend set to false. We don't need the legend anymore because we are b reaking the plot, the line plot into multiple parts and we no longer need to distinguis h between different edge groups in one plot so we don't need a legend set it to fals e and thus there are subplots so we set it to true and the layout is 2 by 2 so we're go ing to have something like this, where we have two by two, have four plots, four sub plots positioned in a two by two layout. Let's do it. The figure size is set into 10 by 10 , that's in terms of inches, but you can change the unit. And of course the size. So he re's what you get the subplots for each subplot corresponds to an age group. It sho ws the test rate of course per 100 000 for that age group. You can see the titles for e ach subplot are positioned. and you can see the titles for each subplot are positione d. So here's an exercise for you to go to chat GPT and see how you can change the u

nit of the thick size from inch to, I don't know, for example, centimeters.

30:09
Okay, chaining or method chaining. We talked about method chaining in the previou s lecture. That's when we call a function and we provide the output of that function a s the input of the next function. And we can keep doing that. That's called chaining. You can do that when you're visualizing the data as well. So here's an example morta lity underlined data that query is going to query our data is going to select all the da ta points that correspond to these particular years 190 and 2018 and then for these t wo years is going to pivot our data frame that is going to create a wide representati on of the death rates, these are the values that are going to be captured and the col umns will be the years so the resulting data frame will have two columns one corresp onds to 190 the other one corresponds to 2018 and the index is age group so the ro ws are age groups so this new data frame has four rows corresponding to different a ge groups and two columns corresponding to different years and there for these two years 190 and 2018 and of course we have the death rates as the values. This is plott ed as a bar plot and as a horizontal bar plot, bar h, horizontal bar plot. So we have d one meta-chain, we have called a query function, the resulting data frame has been passed to the pivot function, that's been pivoted to create a wide representation of t he data, and that is passed to the plot function to plot. And this is the result. We hav e the death rates visualized for different age groups and these two years 190 and 20 08. Here's another example of method chaining for visualization. We're calling well fi rst of all we are grouping our mortality data by year. For each year, we are calculatin g the mean, median, and standard deviation of the death rate. The mean, the averag e of the death rate, corresponds to, for example, 190. And so on and so forth. They'r e calculated, then the group data is plotted as you can see here by setting the Y labe l to whatever. And you can see the plot shows the mean, median and standard devia tion, which are the columns of the new data frame or the grouped data. And yeah, s o we have mean, median, standard deviation for different age groups, for different, s orry, not for different age groups, for different years. Because you grouped your data based on year. And then you displayed it. So for each year, we have these values, me an, median and standard deviation, sure calculated here. And they are stored in the group data that is displayed here.

33:17

All right, so we talked about creating different visualizations, basic visualizations usin g the plot function of pandas. Now I want to see how we can create similar visualizati ons using C1. So again, we need to import the libraries that we need. We need to im port pandas still because we're all going to work with the data frames. And on top of that we also need seaborn because we're going to use seaborn for visualization. As y ou can see we're using sns handle to refer to the seaborn module or library and so y ou're going to see these sns repeated in the code that you're going to see This refers to the seaborn library. Again, get the data. We're reading mortality prepped PKL, whi ch is a PQL file. Read it into the memory, create data frame, and that data frame is m ortality data. We're gonna use this data frame.

34:27
Here's a line plot that is created using Seaborn. So sns.relplot. Relplot stands for relat ional plot. It's a category of plots, that shows relationship between variables. So relpl ot, the type of the plot, or the kind of the plot is line. so this relational plot is a line pl ot and the data is mortality data it's going to show the relationship between these t wo variables year and death rate X and Y and the hue is age group. So here shows th e different categories of the data that we are visualizing. In this case we are visualizin g death rate against year but the values of death rate correspond to different age gr oups and we want to distinguish between these age groups or among these age gro ups that's why we use hue equals age groups and this is going to create a legend th at specifies different colors or is going to denote different age groups by different co lors as depicted in the graph different age groups by different colors as depicted in t he graph. Similarly, we can create subplots for each subplot corresponds to an age g roup here. So we are setting the hue to age group and we are setting the column to be age group and col-wrap to be two which means we're going to plot each age gro up on a separate column and wrap of the two columns. So we're gonna get two sub plots in each row. And again, we're setting the legend to false because we no longer need a legend since we have divided our original plot to subplots. And you can see t hat age group equals one to four, edge group equals five to nine, are used as the de fault titles to specify different subplots.

36:43
Set the pick X limit and Y limit. So here's an example setting these parameters. SNS.li neplot is going to create a line plot. Data equals mortality data of course sets the dat

a. X equals year and Y equals death rate. Hue, you edge group is going to create a li ne plot that shows how death rate changes over time or years, and it's going to disti nguish among different edge groups. So these values are set here after the creation of the plot, we're using this handle ax to refer to the plot so ax.set is going to chang e the attributes of the plot that was created here and it sets the title to this y label to this and then it sets the x-thicks that's going to show the, specify these values on the x-axis, so the x-thick values. And setting them, creating them actually from this range between 1910 to 1931 with the step of 2. So it starts from 1910 and the next one, the value is 1912 then 1914 and then goes all the way up to 1930. We also specifying the X limit and the Y limit to be contained within the ranges are specified here and here and any points outside these ranges will be excluded. Yeah so this is how we can set these attributes using the handle reference to the plot that's created here. You can al so set the background style, which is a simple thing to do. Set style, blah. But this is s omething that you can try yourself and see how different styles might work better.

38:49
Once you created a plot, you may need to save it because maybe you want to includ e it in a report or post it on a website or somewhere. So you want to reuse the figure . You don't want to re-execute the code each time you need the figure. To save that, save the figure, you can use different functions you can you can use different format s you can save it as a PNG, SPG or PDF using save figure. So first you need to get the figure and then save it. So call the get figure and then call the save figure again we a re doing chaining here. But we call this function from the using the reference to the plot. So here's your plot ax equals blah you create the plot and then using you use t he reference to the plot to set some set the attributes and then to save it. This is this is the focus of this slide. ax.get figure gets the figure or retreats the figure and then s aves it into the location specified here. In this case, the location is on the figures dire ctory. You can see it here in the photo. So the current directory has been seaborn. So the Jupyter notebook file has been, IPYMB file has been executed from the seab1 dir ectory or folder and therefore figures slash blah is going to store the figure under thi s directory or this folder with this name.

40:13
Scatter plot. Similar to the line plot you can also create a scatter plot again using the rel plot, relational plot. And this time the type is scatter, the kind is scatter. And agai

n it shows the relationship between year and death rate across different age groups. Bar plot is the next plot that we explore using Seaborn. We have already seen how w e can create this in pandas. Now we're looking at it in Seaborn domain. So using Sea born library to create a barplot. Similarly, oh, this time we're calling the catplot or cat egorical plots, that cat plot or categorical plots. Cat stands for categorical. Again, it's a subcategory of different types of, or subclass of different types of visualization func tions used in Seaborn. There was a rel plot for relational, for showing relationships, a nd there's cat plot that is used for categorical visualizations. As to create a bar plot u sing categorical data select to select the mortality data for year 1950 and 200 again t his is a tuple here it's not a range and to display the mean and confidence interval of death rate using bar chart. Basically what this does is create a bar chart and then this parameter here Lbar is set to CI50. It shows where the mean or the average of the po pulation or average death rate of the population is the real population that your dat a is going to be representative of. So the real mean, the mean of the population falls within the range that is specified here by this line. So this line shows the range within which the real mean of the population is going to fall and the confidence for such cl aim is 50%. So there's a 50% confidence that this, that the mean of the population, t he real mean is going to fall with this within this range and of course 50% is not high this just for demonstration usually we consider our numbers like 95 50 is too low to even be considered but this is just for the sake of demonstration so this shows the c onfidence interval and it's not the focus of this lecture but just for you to know what it does.

43:29
Okay Yeah, so there's nothing new about this one. And again, the popular box plot. As I said, it's a very informative plot. Gives us information about the distribution of th e data, the first and second and third quartile, the outliers, and min, max. So it gives us plenty of information about data and there is another example of creating it using seaborn this time. So catplot function is used by setting the kind to box. It's going to create a boxplot and there is a query here again that specifies we are going to displa y data, the distribution of the data between 1915 and 1920, so all the other data poi nts will be excluded and then the information that all the death rates related to these years are displayed as you can see in the plot. Again you can make your boxplot to b e displayed horizontally by setting the orient to H. This is going to create a horizonta l plot. Histogram similar to Pand's histograms, you can also create a histogram using

C1. Sns displot, dis stands for distribution, it's going to show the distribution of your parameter of interest in this case death rate, x equal death rate, and it's going to use a histogram to show the distribution and you can again see the distribution as befor e. You can also change the size of the beans as you wish. Here's an example you can change the size of the beans and here we have set the beans to 8 which means we're going to have 8 beans which is going to change the size of the beans to 250. When I say the size of the beans we refer to the widths of the beans here. So when we be 25 0 in the histogram.

45:54
Another type of visualization in C1 that concerns the distribution of the data is kerne l distribution estimate that displays a density function. It's not the PDF or probability density function but it's an estimate of that. They're not mathematically the same but for the purpose of this lecture you can consider them as similar. So it again shows th e distribution of your data and to generate it you need to use the, you need to set th e kind to be KD. Simple as that. And again the function is this.distribution.plot. You c an also display the empirical cumulative distribution function of your variable of inter est, in this case, death rate. And here we have multiple ECDFs. Each of them correspo nd to an age group and show the cumulative distribution of death rate. So the x-axis represents the values of the variable, the y-axis represents the proportion of data poi nts that have a value less than or equal to the corresponding x-values, so it's cumulat ive. And the plot is useful for visualizing the distribution of variable and identifying i mportant features such as a median and range of the data. We can overlay a KDE ov er a histogram, simply by setting a parameter KDE to true in the distribution plot. So there's a parameter KDE there, then we can set it to true while we are creating a hist ogram. So essentially we're creating a histogram here, but on top of the histogram, we're also showing the KDE of the variable of interest, in this case, the rate. So you c an see both of them here in one graph, one plot. Here's another example where we are creating a KDE or a K-distribution estimator. And that is again showing the area below the graph and below the plots and sets the column to be age group so we are creating the subplots based on age groups in other words and the call wrap is set to 2 so again we have two subplots in each row and the height is set to 3, the legend is set to false, we don't need the legend because we are separating the plots based on the age groups, there is no need for legend. But these are all the improvements that we can make to make our visualizations more readable. visualizations more readable.

49:29
Sometimes you find an interesting pattern on a graph or a plot that you want to hig hlight. Then you can annotate your plot. The best way to annotate is definitely not th rough coding. It's tedious, it's confusing and it's not the easiest way to do for sure. it is possible so here we can just see how we can do that through annotate function. A X is the reference to the plot that's been created here as a line plot. We are creating annotation we are setting some text so this is the title Spanish Flu which highlights t here has been a Spanish flu in this year in 1925 apparently sorry 1918 and there has been a spike here so this is what motivated us for instance to create annotation to hi ghlight this year or highlight this point that there has been Spanish flu here that's wh y we have a big spike across all different age groups and this is how we are doing th e annotation we're setting the text to be Spanish flu pandemic and we're setting the position of the end of the arrow which is 1918 1650 this is this point and the start of the text 1925 to 190 where this is where the text is started and what else the face col or of the arrow what color it is, the width of it, the head width, the head length and t hose sort of things. So this just shows the attributes of the arrow. Again, I don't reco mmend it and it's not fun to do annotation this way. But you can do it, so you can pr actice and learn how to do it but it's not something that it's not the best way to ann otate a graph

51:29
okay let's see what happens here we have created a figure of this size and the inches and we have two handles to the figure fig and x equals plt the subplots then we are creating a subplot using creating a line plot using the line plot function of the seabor n and the hue is age group so we're distinguishing among different age groups and we're setting the, we're using the X, the reference to the figure to set the title, the la bel, Y label, X sticks and X limit. So you can see that AX is used to make changes to t he display of the SNS plot is because Seaborn is using matplotlib so this part is in thi s part we're using matplotlib you can see matplotlib.pyplot is imported as plt and th en plt that's how plot is created a figure so when we use ax.set blah this is going to b e reflected into the plot that is created and positioned on the figure. So figure in oth er words is like a container, like a canvas that can contain multiple plots. And then ax .set is going to set these attributes for the plot that has been created on the figure. S o there's a connection here that has been made indirectly behind the scene. I know it

's confusing and it's a terrible design in fact but this is how whatever it is it's designe d it like that so matplotlib once created once the figure is created by matplotlib seab orn is using the figure that is created by matplotlib to you know to render its own pl ot and therefore you can get access to the, use the reference to the figure that is cre ated by matplotlib to manipulate the plot that is created by Seaborn. Amazing desig n, I know. But it's like a kind of a context dependency between these two libraries, w hich could lead to error of course. Anyways and how it works and then you have ax.ti ck prompts and you are setting the what you call it the orientation of the label, of th e X labels to 45 degrees. So rotated by 45 degrees. These are the X, X tick labels in ot her words. And they are rotated 45 degrees and eventually we're saving the figure. S o this was an interesting example.

54:49
And the last example that we're going to explore in this lecture is to customize the p roperties of the subplots after creating them. So once the subplots are created we ca n go through them, we can iterate over them and customize their attributes. How ca n we do that? Let's see. g equals sns.relplot is going to give us a reference to a relati onal plot of type line, so it's going to be a line plot and with all the attributes that ar e set here and of course these are subplots. So you have two columns and two rows, four subplots and the specifying the height and spec ratio blah and of course each s ubplot corresponds to one specific edge group as we have seen before so nothing n ew so far and what is new however is this part we are creating an age group a list of age groups that is composed of the unique age groups in the age group column of t he mortality data. So mortality data age group is going to extract all the age groups and then we're going to drop the duplicates which gives us a list of unique age grou ps. And once converted to a list it's going to give us a unique age groups so age gro up in other words will age groups in other words is a list that contains you know 0 to 1 years 5 to 9 years 10 to 14 years and 15 to 19 years we're going to use this to extra ct their subtitles or the titles of the subplots. But before that, we need to be able to it erate over the subplots. How do we do that? We need a handle to the subplots. How do we get that handle? We can extract it from g. g is a handle to the plot that contai ns the subplots. So g.axis will give us all the subplots that we need or handles the su bplots that we need. But they come in a matrix structure. So it will come like this. So you have your axis 0 here. I just make it short and put the numbers. Axis 1 xs2, xs3. T hese are the references or the handles to your xs1, two, one, sorry, grab the laser poi

nter, one, two, three, four, okay? However, we want to be able to iterate over this in s equence, in one for loop, in a way that we can map each of these into their correspo nding titles in the age group list. So to make it easy we're going to flatten this. This is what this line, this part does. xs.flat is going to give us a flattened representation of t he handles to all these four xs. So again your xs1 goes here, your first axis goes here, that's index 0, the second one index 1 goes here, third one index 2 goes here and th e fourth one index 3 goes here. So you can iterate over these and then you can use a variable x to refer to each of these in each iteration. So in your first iteration of the lo op, in the first iteration of the loop, x refers to the first subplot. In the second iteratio n, it refers to the second subplot. In the third iteration, it refers to the third subplot. And in the last iteration, it refers to the third subplot and in the last iteration refers t o the third subplot. And what else? You can also see there is index here. So index is a number. So zero for X zero, one for X one, two for X two and three for X two. We use this index to extract the appropriate or the corresponding or the respective title of th e subplot as specified here. So age group of index is going to give us the whatever t he index number is in the first iteration 0 so it's going to extract the title that is assoc iated with the first axis, axis index 0 and so on so forth. And in a similar way the rest of the attributes are updated. So x.setTitle sets the title of the first subplot in the first iteration, second subplot in the second iteration and so on so forth. SetYLabel sets th e label of the Y label of the subplot and this one is the same for all the subplots so w e don't need to distinguish between different subplots is the same we don't need to use index to extract anything. It's the same. Similarly, the XTec is the same. Just speci fy the specified range. The values between 1910 to 1930 with the step of 2. And the r otation is 45 degrees for all the X ticks of all the subplots. So the only attribute that i s different for different subplots is the subtitle, is the title, which is extracted from, yo u know, the age group list based on the indices. If it was confusing, please repeat tha t, replay this part and in the recording and listen again and try to do it yourself so yo u understand steps if you need more explanation or clarification please feel free to di scuss with your during the workshops or help sessions or and or use ChatGPT.

01:01:34
Last but not least there is a beautiful movie or video here. It's not a movie actually. T hat summarizes some of the main capabilities of Plotly and Bokeh and other libraries that you can use for data visualization and they provide some what you call it interac tive functionalities and high quality visualization. So I strongly recommend watching

this video. It's a good summary. And this is the end of this lecture.